



Smart Grid Energy Monitoring & Billing: A Lambda-Architecture Data Pipeline

Mini Project Assessment

EC8203 Applied Big Data Engineering

A mini project report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

by

Sewvandi M.A.K.	- EG/2021/4808
Peiris P.R.S.	- EG/2021/4706

September 2026

Contents

1. Introduction and Use Case.....	3
1.1 Business question and derived requirements.....	3
1.2 Data sources (simulated, Python).....	3
1.3 Simulated clock and assumptions.....	3
2. Architecture Decision: Lambda vs Kappa.....	4
2.1 Why Lambda fits this problem.....	4
2.2 The rejected alternative: Kappa.....	4
2.3 Trade-offs we accepted and how we contain them.....	5
3. System Architecture	6
4. Technology Stack and Justification.....	6
5. Implementation.....	7
5.1 Ingestion.....	7
5.2 Speed layer	7
5.3 Batch layer and orchestration	7
5.4 Storage and serving layer	8
6. Observability Design.....	9
6.1 Failure drill: stopping the streaming source	10
7. Testing and Reproducibility	11
8. Results	11
8.1 Live dashboard and real-time API.....	11
8.2 Daily billing and solar-contribution report (batch view).....	12
8.3 Lambda merge: provisional bills during the day.....	13
8.4 Speed vs batch reconciliation and data quality.....	14
8.5 Alerts, orchestration and monitoring.....	14
9. Limitations, Trade-offs and Production-Scale Changes.....	15
10. Conclusion.....	16
Individual contributions	16

1. Introduction and Use Case

A utility company wants real-time visibility into grid load and the renewable (solar) contribution from household smart meters, reconciled once a day against tariff and billing reference data published by its billing system. We implemented **Use Case 3** with both required source types: a continuous smart-meter stream and a once-per-day tariff extract.

1.1 Business question and derived requirements

Business question: “What is the current grid load and renewable contribution by zone, and what will each household's bill look like once daily tariff data is applied to their consumption?” We decomposed it into functional (FR) and non-functional (NFR) requirements, which drive every later decision:

ID	Requirement	Where it is met
FR1	Live load, solar generation and renewable % per grid zone	Speed layer → /api/grid/live, dashboard
FR2	Alert when renewable contribution is considerably low	LOW_RENEWABLE rule (daylight only)
FR3	Daily per-household bill and solar-contribution report once the tariff arrives	Batch layer → daily_billing_report, CSV/HTML report files
FR4	An estimate of today's bill before the day closes	Lambda merge → /api/billing/projection
FR5	Tolerate dirty data in both feeds	Quarantine, dedup, tariff validation + fallback
NFR1 Latency	Live view within seconds; bills within minutes of the tariff file	10 s micro-batches; DAG polls every 60 s
NFR2 Correctness / replay	Bills must be exact, reproducible and re-runnable	Immutable Parquet lake + idempotent per-day overwrite
NFR3 Consistency	Readers never see half a day; speed and batch comparable	Single-transaction day write; reconciliation table
NFR4 Observability	Detect and diagnose failures in any stage	JSON logs, heartbeats, 7 alert rules, Prometheus
NFR5 Cost / operability	Runs on one laptop with one command	Docker Compose, 9 containers

1.2 Data sources (simulated, Python)

Streaming source: 24 households in 4 zones each emit a JSON reading every 2 s — event_id, meter_id, household_id, power_consumption_kwh, solar_generation_kwh, grid_zone, timestamp. Consumption follows morning/evening peaks with noise and ~1% demand spikes; solar follows a daylight curve scaled by zone capacity (South has few panels) and a deterministic daily cloud factor, and each simulated day one zone suffers an overcast spell. ~1% of events are deliberately dirty (missing field, negative or impossible value, unparsable timestamp, or an exact duplicate simulating at-least-once redelivery).

Daily-batch source: 20 s after each simulated day ends it publishes that day's end-of-day tariff extract as a CSV file (household_id, tariff_rate, billing_tier, subsidy_flag, sim_day, published_at). The file is written to a temporary name, atomically renamed, and only then is a completion marker written. Tier and subsidy are fixed per household and the daily rate drift is seeded by (household, day), so regenerating a day yields an identical file. About one file in three contains an invalid rate and some contain a duplicated household row; on restart the source back-fills any missing days.

1.3 Simulated clock and assumptions

- **1 simulated day = 300 s of real time** (configurable). Day fraction 0.5 is simulated noon; daylight for alerting is fraction 0.40–0.60.

- Day numbers come from ONE epoch stored in Postgres (`sim_clock`) when the database is created: $\text{sim_day} = \text{floor}((\text{event_time} - \text{epoch}) / 300)$. Using event time (not processing time) means a late event is still attributed to the correct day and every container agrees on day boundaries.
- Billing: $\text{bill} = \max(\text{consumption} - \text{solar}, 0) \times \text{tariff_rate} \times (1 - 15\% \text{ if subsidised})$; no export credit. Renewable contribution = $\text{solar} \div \text{demand}$, capped at 100%. Currency LKR. All data is synthetic.

2. Architecture Decision: Lambda vs Kappa

Both architectures ingest the same immutable event log. **Kappa** has one stream-processing code path and recomputes history by replaying the log through it. **Lambda** keeps a speed layer for low-latency, approximate results and a batch layer that periodically recomputes exact results from the master dataset, with the serving layer merging both. We evaluated them against the four dimensions the rubric names:

Dimension	What this use case needs	Lambda	Kappa
Latency	Seconds for the grid view; minutes for bills	Speed layer: ~10–15 s end-to-end (measured). Batch: bill ~1 min after the tariff file	Same stream latency; bills also streaming
Replay / correction	Re-bill one day after a corrected tariff file or a code fix	Re-run one Spark job over one Parquet partition (<code>sim_day=N</code>), idempotent	Replay the retained Kafka log from before day N through a new job version, then swap outputs
Consistency	Exact, auditable, all-or-nothing daily bills; duplicates removed	Global dedup on event_id + single-transaction day overwrite	Needs exactly-once state + careful watermarking to finalise a day
Cost / complexity	Laptop-scale, 2-week build, defensible line by line	Two code paths (mitigated: one engine, shared rule modules)	One code path, but a long-state stream-to-file join

2.1 Why Lambda fits this problem

- **The sources have different natures.** Meter readings are a true stream; the tariff feed is a complete once-a-day snapshot. Lambda lets each be processed in its natural form. In Kappa, the daily file would have to be pushed into Kafka as a degenerate 24-event stream and joined against the meter stream with day-long state and a watermark long enough to cover a file that arrives after the day ends.
- **Billing is correctness-critical and must be replayable.** A bill must be reproducible from source data. Our batch layer recomputes each household's total from the immutable lake with a global dedup on event_id and replaces the day's rows in one transaction; re-running it for the same day gives identical bills. The speed layer is explicitly allowed to be approximate — it only dedups within a micro-batch and keeps additive running totals, which double-count if Kafka redelivers after a crash.
- **We can measure the difference, not just claim it.** For every billed day the batch job writes speed-vs-batch drift to batch_reconciliation. Across the 5 simulated days of our run, drift ranged from 0.00% to 0.27% and the batch layer removed 15 duplicate events that the speed layer had counted (section 8).
- **Cheap, scoped reprocessing.** Correcting day N means re-reading one small partition, not the whole log; the raw lake also outlives Kafka's 7-day retention, which a Kappa design would need to extend indefinitely (or add tiered storage) to reprocess older days.
- **Orchestration has a natural home.** The batch layer is a dependency chain (wait for file → Spark → report → re-check alerts) that benefits from Airflow's retries, backfill and UI.

2.2 The rejected alternative: Kappa

Kappa would win if **all** inputs were naturally streams, if bills were needed continuously rather than per day, or if keeping two code paths consistent were the dominant cost. Its genuine advantages are one code path, no batch/speed divergence by construction, and simpler operations. We rejected it here because (1) the daily tariff snapshot is not a natural stream, (2) finalising a day exactly in a stream requires exactly-once state and a watermark tuned to a file that arrives after the day ends, and (3) correcting one day would mean replaying the

log through a new job version and swapping outputs — disproportionate for a one-day fix. We keep the replayable Kafka log anyway, so a future move towards Kappa is possible.

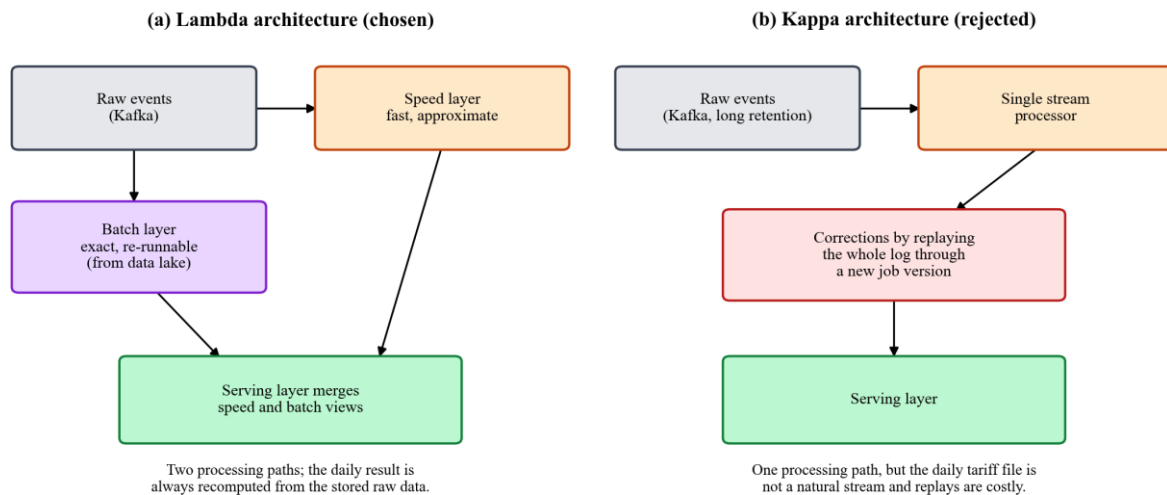


Figure 1. Lambda (chosen) versus Kappa (rejected) for this use case.

2.3 Trade-offs we accepted and how we contain them

- **Two code paths that could diverge.** Mitigated by using Spark for both layers and by sharing the rules: one validation module and one billing-formula module, used by the batch job and by the API's projection. The end-to-end smoke test re-checks every produced bill against the billing formula.
- **The speed view can be briefly wrong** (duplicates, late events beyond the 1-minute watermark). Accepted because it is only used for monitoring and provisional estimates, which the API labels provisional; final bills come from the batch layer only.
- **Bills are only available after the day closes (plus ~1 min).** Accepted: the business asks for daily bills, and /api/billing/projection covers the intra-day need.

3. System Architecture

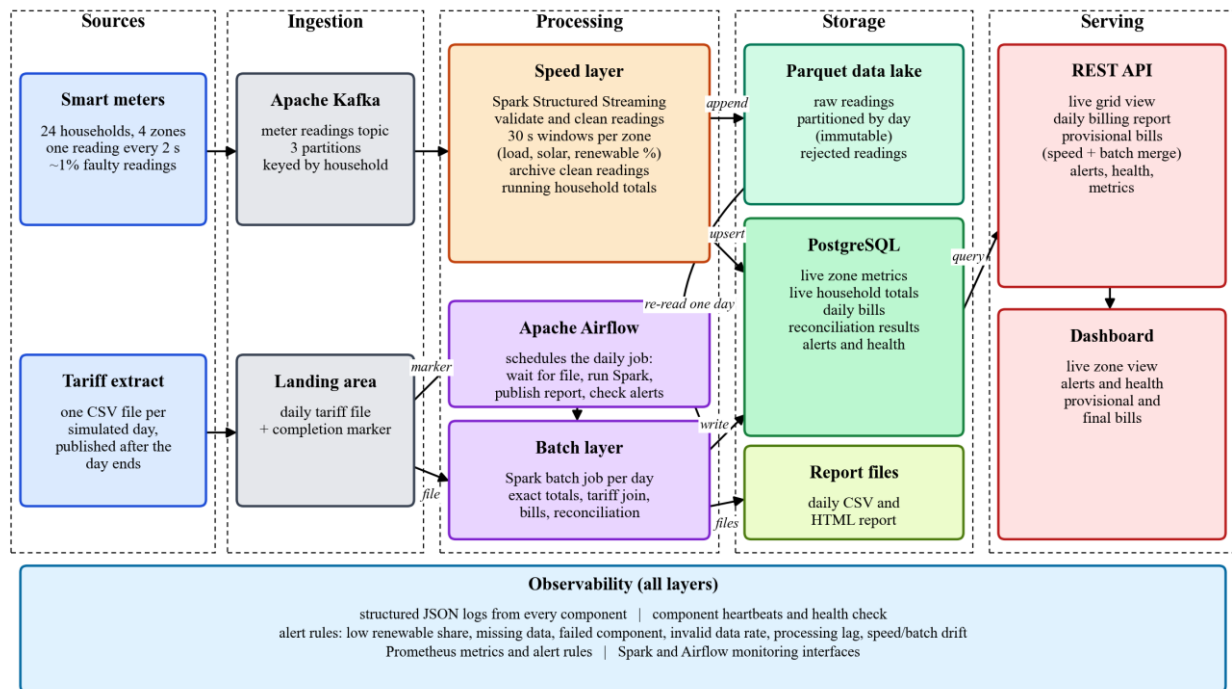


Figure 2. End-to-end architecture across the ingestion, processing, storage and serving layers, with observability as a cross-cutting concern.

Data flow. (1) The producer writes readings to Kafka topic smart-meter-readings (3 partitions, keyed by household_id). (2) The speed layer runs two Structured Streaming queries from checkpointed offsets: a windowed aggregate per zone that is upserted into live_zone_metrics, and a foreachBatch sink that quarantines invalid events, appends clean events to the Parquet lake partitioned by sim_day, adds per-household running totals and records data-quality/lag metrics. (3) After day N ends, the tariff source drops its CSV and completion marker. (4) The Airflow DAG detects the oldest unbilled finished day, waits for the marker, runs the Spark batch job and publishes report files. (5) The batch job writes the whole day to PostgreSQL in one transaction. (6) FastAPI serves the speed view, the batch view and their merge; the dashboard, the report files and Prometheus consume it.

4. Technology Stack and Justification

Layer	Choice	Why this fits the use case	Alternatives considered
Ingestion	Apache Kafka 3.7 (KRaft, 1 broker), 3 partitions, key = household_id, acks=all, 7-day retention	Durable, replayable buffer for continuous telemetry; keying keeps each household's readings ordered while spreading 24 meters across partitions for parallel consumers; retention lets the speed layer catch up after an outage (it starts from checkpointed offsets)	RabbitMQ (no replay/offsets); Kafka+ZooKeeper (KRaft removes a component)
Stream processing	Spark Structured Streaming 3.5	Event-time windows with watermarks for late meter data, checkpointed exactly-once source offsets, micro-batch foreachBatch for multi-sink writes, and the same DataFrame API as the batch job — one engine to learn and defend	Apache Storm (tuple-at-a-time, no shared batch API); Flink (not in the preferred stack; lower latency we do not need)
Batch processing	Spark (local[2]) via spark-submit	Scans one Parquet partition per day, global dedup, joins with the tariff file; scales to a cluster unchanged	Plain pandas (would not scale; different API from the speed layer)

Layer	Choice	Why this fits the use case	Alternatives considered
Orchestration	Apache Airflow 2.9	Daily dependency chain with a file sensor, retries, failure callbacks, backfill (look-back of 3 days) and a UI for demo/diagnosis	Cron (no dependencies, retries or visibility)
Master dataset	Parquet data lake partitioned by sim_day	Immutable, columnar, cheap; partition pruning means a day's recompute reads only that day	Keeping raw events in Postgres (row store, grows unbounded)
Serving store	PostgreSQL 16	Outputs are small and relational and are queried with filters, joins (projection = live totals × latest tariff) and aggregates; transactions and upserts make both layers idempotent	Cassandra (write-optimised, no joins/transactions — our reads need both); HDFS/S3 alone (not a low-latency query store)
Serving API / UI	FastAPI + static HTML dashboard	Typed endpoints and generated OpenAPI docs; dashboard uses only the public API	Grafana-only (cannot express the billing merge)
Observability	JSON logging, Postgres heartbeats, Python rule engine, Prometheus	Business- and pipeline-aware rules (daylight-aware renewable rule, per-component staleness budgets) plus a standard metrics/alerting stack	ELK/Grafana (heavier than needed for one machine)
Packaging	Docker Compose (9 services)	One-command reproducible environment; images built from official bases (Apache Kafka, Apache Airflow, Python + PySpark)	Manual installs

5. Implementation

5.1 Ingestion

- The producer creates the topic explicitly with `KafkaAdminClient` (3 partitions), retries while the broker starts, uses `acks=all`, `max_in_flight_requests_per_connection=1` (ordering on retry) and delivery callbacks that count acknowledged/failed sends; it flushes and closes cleanly on `SIGTERM` and exposes Prometheus counters on port 8001.
- The batch source never exposes a partial file (temp file + atomic rename + marker written last), is deterministic for replays, and back-fills missed days after a restart.

5.2 Speed layer

- Reads Kafka from earliest on first start and from checkpointed offsets afterwards (checkpoints live on the lake volume), so nothing produced before Spark is ready is lost and restarts resume where they stopped.
- **Cleaning:** every event is parsed against an explicit schema and validated by the shared data-quality rules (missing field, bad timestamp, unknown zone, negative, above 50 kWh). Invalid events are written with their reason to a quarantine Parquet path — never silently dropped.
- **Windowing and aggregation:** clean events → 1-minute watermark → 30 s windows sliding every 15 s per zone → load, solar, renewable % and distinct households → idempotent upsert into `live_zone_metrics` (update output mode).
- **Master dataset and enrichment:** in the second query each micro-batch is deduplicated on `event_id`, tagged with its event-time `sim_day` and appended to that day's partition of the raw-event lake; per-household running totals for the day are added into `live_household_consumption`; volume, invalid counts by reason, duplicates and event-to-sink lag go to `stream_quality_metrics`, and a heartbeat to `pipeline_health`.

5.3 Batch layer and orchestration

- Reads only partition `sim_day=N`, removes duplicates **globally** on `event_id`, and computes exact totals: consumption, solar, net, solar export, solar-contribution %, peak reading and reading count per household.

- Reads the tariff CSV with an explicit all-string schema, rejects rows with blank, non-numeric or non-positive rates or unknown tiers, keeps one row per household, and **falls back to the household's last valid tariff** from earlier days (flagged `tariff_source = fallback_previous_day`). Households with no tariff at all are logged and counted, not billed with a guess.
- Joins consumption with tariffs, applies the billing formula and writes `household_daily_consumption`, `tariff_reference`, `daily_billing_report` and `batch_reconciliation` with delete-then-insert for day N **in one transaction** — the day appears atomically and re-runs are idempotent. Each run has a `run_id` that appears in every log line and in the bills.
- The Airflow DAG `daily_batch_pipeline` runs every 60 s: `resolve_sim_day` (ShortCircuit: oldest finished, unbilled day with raw data, 3-day look-back) → `wait_for_tariff_file` (sensor) → `run_batch_layer` (spark-submit) → `publish_report` (CSV + HTML) → `run_alert_check` → `notify_complete`. Retries (2×20 s) and an `on_failure_callback` that raises a CRITICAL PIPELINE_TASK_FAILED alert make failures visible. Polling + ShortCircuit is used instead of a daily cron because with the compressed clock days end every 5 minutes at arbitrary offsets.

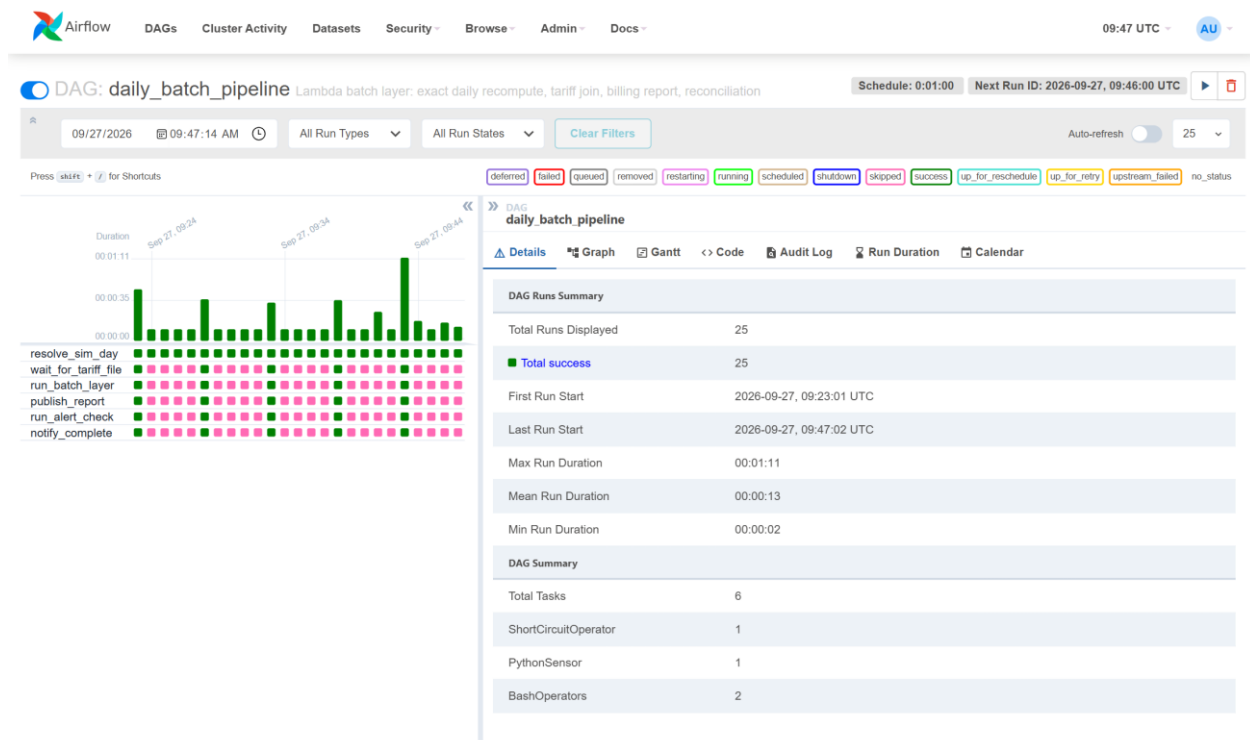


Figure 3. Airflow grid view of `daily_batch_pipeline`: full runs (green) bill a finished simulated day; polls in between short-circuit and skip downstream tasks (pink).

5.4 Storage and serving layer

Table / data	Written by	Purpose
Parquet raw events, partitioned by <code>sim_day</code>	Speed layer	Immutable master dataset for the batch layer
Parquet quarantine, partitioned by invalid reason	Speed layer	Rejected events kept for diagnosis
<code>live_zone_metrics</code>	Speed layer	Windowed load / solar / renewable % per zone (6 h retention)
<code>live_household_consumption</code>	Speed layer	Approximate running totals for the current day
<code>stream_quality_metrics</code>	Speed layer	Per micro-batch volume, invalid/duplicate counts, lag
<code>household_daily_consumption</code> , <code>tariff_reference</code> , <code>daily_billing_report</code>	Batch layer	Exact daily view and bills
<code>batch_reconciliation</code>	Batch layer	Speed vs batch drift and data-quality counters per day
alerts, <code>pipeline_health</code> , <code>sim_clock</code>	Observability / shared	Alert lifecycle, heartbeats, shared epoch

Endpoint	View	Answers
GET /api/grid/live, /api/grid/history	Speed	Current load, solar and renewable % per zone; 15-minute history
GET /api/households/live	Speed	Today's running consumption per household
GET /api/billing/daily, /summary, /days	Batch	Per-household bill and solar contribution; zone roll-up; billed days
GET /api/billing/projection	Merge	Provisional bill so far today = speed consumption × latest batch-validated tariff, beside the last final bill
GET /api/reconciliation	Batch	Speed-vs-batch drift and data-quality counters
GET /api/alerts, /api/pipeline/status, /health, /metrics	Observability	Open/resolved alerts, heartbeats, stream quality, Prometheus metrics

The latest-window query deliberately returns the latest **complete** window per zone (a still-open window holds only a few seconds of data and would under-report load). Database outages are turned into logged HTTP 503 responses rather than crashes. The API also serves the dashboard and generated OpenAPI docs.

6. Observability Design

The goal is that any failure — a dead producer, a stuck Spark query, a crashed batch job, a bad input file — is **detected** automatically and can be **diagnosed** from a single place. We instrument four signal types, and use shared correlation IDs for end-to-end tracing:

Signal	How it is produced	Why
Structured logs	Every component logs one JSON object per event (time, level, component, msg + fields) to standard output and a per-component log file; correlation keys sim_day, batch_id, run_id, event_id, household_id	Trace one day or one batch run across producer → Spark → Airflow → API; machine-parseable for a log shipper
Heartbeats	Each long-running component upserts pipeline_health (status + detail); each component has its own staleness budget	Liveness per stage; the batch layer is legitimately silent for a day, so one global threshold would give false alarms
Pipeline metrics	stream_quality_metrics per micro-batch; batch_reconciliation per day; exported by /metrics (refreshed from Postgres on each scrape) and the producer's :8001	Volume, invalid rate by reason, duplicates, event-time lag, speed-vs-batch drift, delivery failures, request latency
Alerts	A Python rule engine every 15 s and after each batch run; Prometheus evaluates the equivalent alert rules	Turns signals into de-duplicated, auto-resolving, optionally webhook-notified alerts

Rule	Condition (default threshold)	Detects
LOW_RENEWABLE (business)	zone renewable % < 10% in the latest complete window, during simulated daylight only	Grid leaning on non-renewable supply (e.g. overcast zone)
NO_DATA	heartbeat older than the component's budget (2 min streaming; 2 days + 2 min for the batch layer)	Dead/stuck producer, Spark query, alerts engine or batch layer
COMPONENT_FAILED	component reported status ≠ OK (e.g. batch job exception)	Crashes that still heartbeat
HIGH_INVALID_RATE	> 5% of events quarantined in 5 min	Upstream schema/sensor problem
HIGH_STREAM_LAG	average event→sink lag > 60 s	Speed layer falling behind
RECONCILIATION_DRIFT	speed – batch / batch > 10% for the last billed day	Speed-layer double counting / data loss
PIPELINE_TASK_FAILED	Airflow on_failure_callback	Batch orchestration failure

Alerts are stored with a partial unique index so that at most one alert per (type, scope) is open; while the condition persists its last_seen_at and value are refreshed, and when it clears the alert is auto-resolved. Every transition is logged, and posted to a Slack/Teams-compatible webhook when one is configured.

Tracing. Every reading carries an event_id, every speed-layer micro-batch a batch_id and every batch-layer run a run_id, and all of them, together with sim_day and household_id, are written into the structured logs and stored with the results (each bill records the run_id that produced it). This lets a single reading, micro-batch or daily billing run be traced end to end from the producer through Kafka, Spark and Airflow to PostgreSQL and the API, which is how a failure is located to the stage where it happened.

6.1 Failure drill: stopping the streaming source

To show that failures are detected and diagnosable, we stopped the producer container at 09:26:21 UTC. The rule engine raised 2 NO_DATA alert(s) (meter_producer, speed_layer) and /health reported the affected components as stale; after restarting the producer the speed layer resumed from its Kafka offsets and every NO_DATA alert auto-resolved without manual action.

Time (UTC)	Observation
09:26:21	docker compose stop meter-producer (streaming source killed)
09:28:23	/health marks meter_producer STALE (last heartbeat 127 s ago); overall DEGRADED
09:28:33	/health marks speed_layer STALE (last heartbeat 122 s ago); overall DEGRADED
09:28:33	Alert NO_DATA (CRITICAL) raised for meter_producer: No heartbeat from meter_producer for 2.2 min (budget 2.0 min)
09:28:43	Alert NO_DATA (CRITICAL) raised for speed_layer: No heartbeat from speed_layer for 2.0 min (budget 2.0 min)
09:28:51	Dashboard screenshot taken (status pill DEGRADED, NO_DATA alerts listed)
09:29:52	docker compose start meter-producer (source restored)
09:30:12	All NO_DATA alerts auto-resolved; /health back to OK

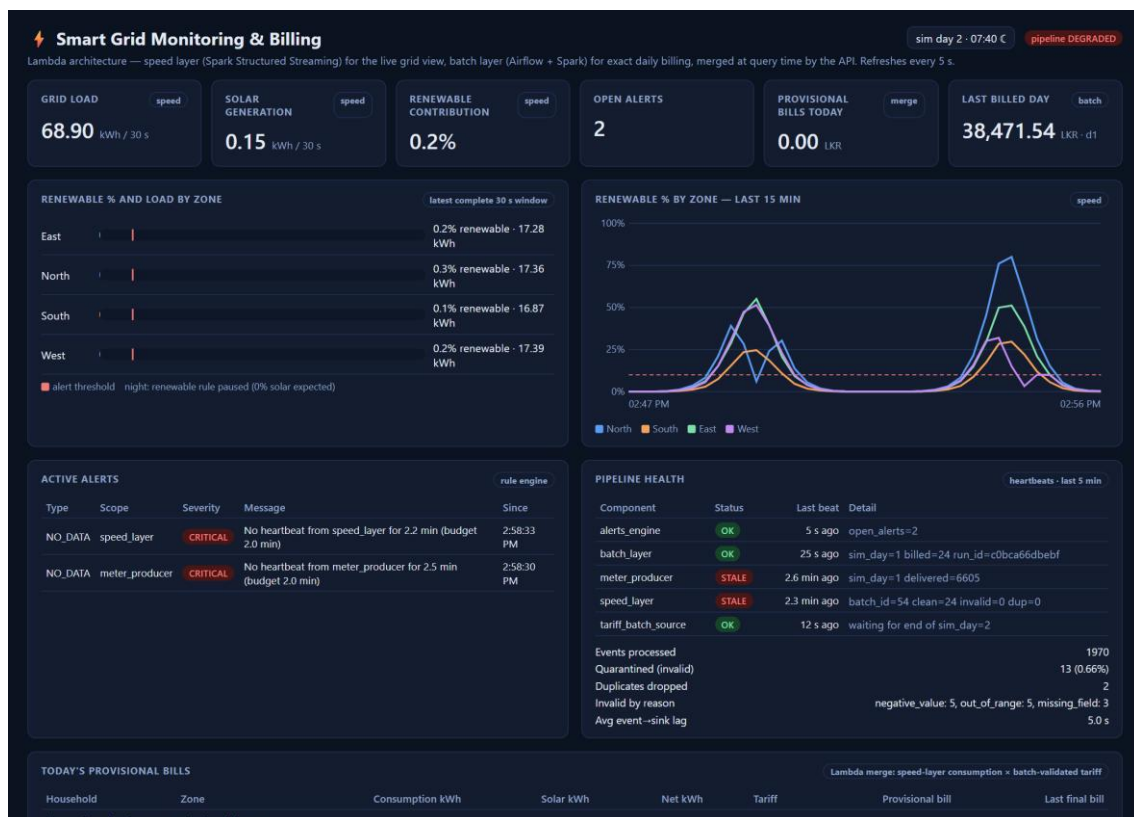


Figure 4. Dashboard during the drill: NO_DATA alerts and stale components.

7. Testing and Reproducibility

- **Unit tests** (no infrastructure needed): data-quality rules, billing formula, simulated clock, both simulators (fields, value ranges, determinism, overcast spell, every dirty kind fails validation, atomic file drop), alert predicates, API endpoints with a stubbed database (projection merge, health staleness, DB-outage 503, metrics) and the report renderer.

collected tests: 44

44 passed in 1.04s

- **End-to-end smoke test** against the running stack: events delivered, live windows for all zones, quarantine working, health check OK, Prometheus rules loaded and targets up, every bill equal to the billing formula, reconciliation present, projection populated.

```
[PASS] ingestion: producer delivered events to Kafka -- 10490 delivered
[PASS] speed layer: live window for all 4 zones -- ['East', 'North', 'South', 'West']
[PASS] speed layer: events processed in last 5 min -- 3679 events
[PASS] processing: dirty events are quarantined -- {"bad_timestamp": 16, "out_of_range": 12, "missing_field": 10, "negative_value": 7}
[PASS] observability: /health reports every component OK -- alerts_engine=OK, batch_layer=OK, meter_producer=OK,
speed_layer=OK, tariff_batch_source=OK
[PASS] observability: /metrics exports pipeline gauges
[PASS] observability: Prometheus alert rules loaded -- 9 rules
[PASS] observability: Prometheus scrape targets up -- meter-producer=up, prometheus=up, serving-api=up
[PASS] batch layer: every bill matches the billing formula -- 24 bills for sim_day 4, mismatches=[]
[PASS] batch layer: speed/batch reconciliation recorded -- drift=0.14%
[PASS] lambda merge: provisional bills for the running day -- 24 households, total 15491.04
```

ALL CHECKS PASSED

- **Reproduce:** start the whole stack with one Docker Compose command, wait about six minutes for the first bill, then run the end-to-end smoke test. All configuration is set through environment variables.

8. Results

The results below come from one continuous run of the stack (5 simulated days, ~25 minutes of real time). Every number was read from the API by an automated capture script.

8.1 Live dashboard and real-time API

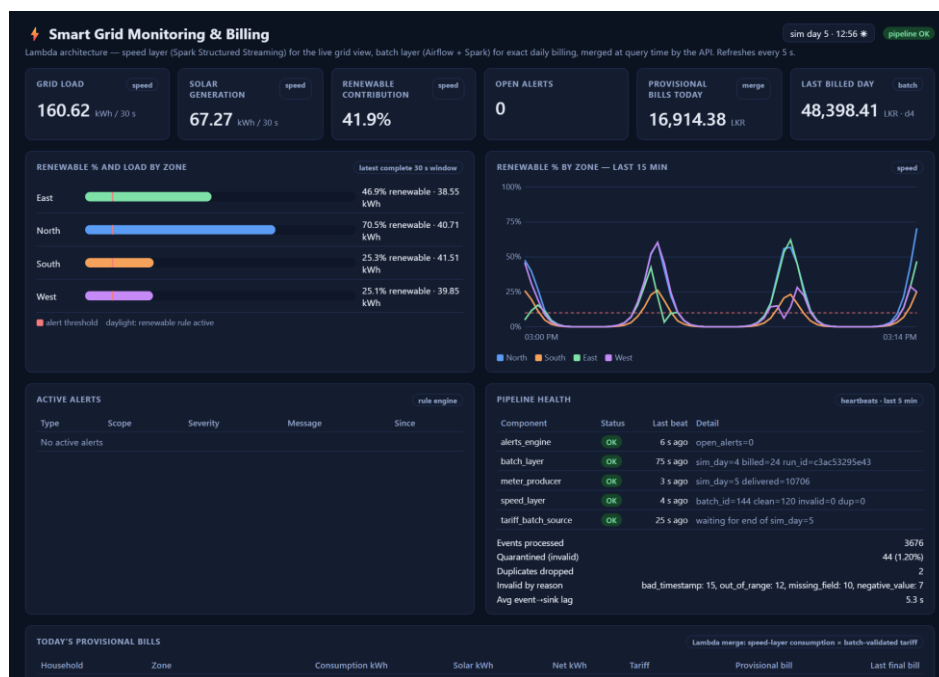


Figure 5. Live dashboard: speed-layer KPIs, per-zone renewable % with the alert threshold, 15-minute history, active alerts and pipeline health.

```

GET http://localhost:8000/api/grid/live

{
  "source": "speed_layer",
  "clock": {
    "sim_day": 5,
    "day_fraction": 0.545,
    "sim_time_of_day": "13:05",
    "daylight": true,
    "simulated_day_seconds": 300
  },
  "zones": [
    {
      "grid_zone": "East",
      "window_start": "2026-09-27T09:44:15+00:00",
      "window_end": "2026-09-27T09:44:45+00:00",
      "avg_consumption_kwh": 0.4331011235955057,
      "avg_solar_kwh": 0.20293258426966293,
      "total_consumption_kwh": 38.54600000000001,
      "total_solar_kwh": 18.061,
      "renewable_pct": 46.85570487210086,
      "event_count": 89,
      "household_count": 6,
      "updated_at": "2026-09-27T09:44:52.107727+00:00",
      "below_threshold": false,
      "low_renewable": false
    },
    {
      "grid_zone": "North",
      "window_start": "2026-09-27T09:44:15+00:00",
      "window_end": "2026-09-27T09:44:45+00:00",
      "avg_consumption_kwh": 0.4626022727272728,
      "avg_solar_kwh": 0.3260795454545455,
      "total_consumption_kwh": 40.709,
      "total_solar_kwh": 28.695,
      "renewable_pct": 70.48809845488712,
      "event_count": 88,
      "household_count": 6,
      "updated_at": "2026-09-27T09:44:52.107727+00:00",
      "below_threshold": false,
      "low_renewable": false
    },
    {
      "grid_zone": "South",
      "window_start": "2026-09-27T09:44:15+00:00",
      "window_end": "2026-09-27T09:44:45+00:00",
      "avg_consumption_kwh": 0.4612333333333333,
      "avg_solar_kwh": 0.1168,
      "total_consumption_kwh": 41.510999999999996,
      "total_solar_kwh": 10.512,
      "renewable_pct": 25.323408253234085,
      "event_count": 90,
      "household_count": 6,
      "updated_at": "2026-09-27T09:44:52.107727+00:00",
      "below_threshold": false,
      "low_renewable": false
    },
    {
      "grid_zone": "West",
      "window_start": "2026-09-27T09:44:15+00:00",
      "window_end": "2026-09-27T09:44:45+00:00",
      "avg_consumption_kwh": 0.44779775280898876,
      ... (truncated)
    }
  ]
}

```

Figure 6. GET /api/grid/live — latest complete window per zone (speed view).

8.2 Daily billing and solar-contribution report (batch view)

For simulated day 4, the batch layer billed 24 households for a total of LKR 48,398.41. Zone roll-up:

Zone	Households	Consumption kWh	Solar kWh	Avg solar contribution %	Billed (LKR)
East	6	567.85	70.06	12.3	12,339.32
North	6	558.27	70.17	12.6	10,960.30
South	6	562.47	25.92	4.6	12,609.07
West	6	552.53	35.68	6.5	12,489.72

First rows of the per-household billing report:

Household	Zone	Tier	Net kWh	Solar %	Tariff	Source	Subsidy	Bill (LKR)
H-000	North	standard	79.86	12.6	26.50	today	no	2,116.32
H-001	South	low	89.34	4.6	17.81	today	no	1,591.13
H-002	East	high	83.32	12.3	31.74	today	no	2,644.42
H-003	West	low	86.44	6.6	17.33	today	no	1,498.01
H-004	North	low	77.51	13.1	18.81	today	no	1,457.96
H-005	South	standard	87.78	4.8	26.20	today	no	2,299.94
H-006	East	low	79.61	12.9	17.52	today	no	1,394.84

Household	Zone	Tier	Net kWh	Solar %	Tariff	Source	Subsidy	Bill (LKR)
H-007	West	high	91.30	6.3	31.24	today	no	2,852.24

Smart Grid - Consolidated Daily Billing & Solar Report

Simulated day 4 · generated 2026-09-27 09:43:50 UTC · batch run c3ac53295e43

Households billed **24** Total billed (LKR) **48,398.41** Consumption (kWh) **2,241.12** Solar generated (kWh) **201.83**

By zone

Zone	Households	Consumption kWh	Solar kWh	Avg solar contribution %	Billed LKR
East	6	567.85	70.06	12.3	12,339.32
North	6	558.27	70.16	12.6	10,960.30
South	6	562.46	25.92	4.6	12,609.07
West	6	552.53	35.68	6.5	12,489.72

Data quality & speed/batch reconciliation

Raw events	Duplicates removed	Speed-layer kWh	Batch kWh	Drift %	Tariff rows	Invalid tariff rows	Fallback tariffs	Unbilled households
3641	4	2244.16	2241.12	+0.14	25	2	1	0

Top 5 bills

Household	Zone	Net kWh	Tariff	Bill LKR
H-007	West	91.30	31.24	2,852.24
H-011	West	85.95	31.52	2,709.14
H-002	East	83.32	31.74	2,644.42
H-018	East	84.66	31.21	2,642.27
H-017	South	91.79	26.20	2,404.79

All households

Household	Zone	Tier	Consumption	Solar	Net	Solar %	Tariff	Tariff source	Subsidy	Bill LKR
H-000	North	residential_standard	91.39	11.53	79.86	12.6	26.50	current_day	no	2,116.32
H-001	South	residential_low	93.64	4.30	89.34	4.6	17.81	current_day	no	1,591.13
H-002	East	residential_high	95.04	11.73	83.32	12.3	31.74	current_day	no	2,644.42
H-003	West	residential_low	92.52	6.08	86.44	6.6	17.33	current_day	no	1,498.01
H-004	North	residential_low	89.18	11.67	77.51	13.1	18.81	current_day	no	1,457.96
H-005	South	residential_standard	92.17	4.38	87.78	4.8	26.20	current_day	no	2,299.94
H-006	East	residential_low	91.43	11.81	79.61	12.9	17.52	current_day	no	1,394.84
H-007	West	residential_high	97.40	6.10	91.30	6.3	31.24	current_day	no	2,852.24
H-008	North	residential_low	94.51	11.43	83.08	12.1	19.18	current_day	no	1,593.46
H-009	South	residential_standard	96.53	4.34	92.19	4.5	25.29	current_day	no	2,331.46
H-010	East	residential_standard	98.40	11.76	86.64	11.9	24.88	fallback_previous_day	no	2,155.50
H-011	West	residential_high	91.69	5.74	85.95	6.3	31.52	current_day	no	2,709.14
H-012	North	residential_standard	97.56	11.97	85.60	12.3	25.39	current_day	no	2,173.33
H-013	South	residential_standard	93.27	4.32	88.95	4.6	25.40	current_day	yes	1,920.50
H-014	East	residential_standard	93.94	11.47	82.46	12.2	24.06	current_day	no	1,984.08
H-015	West	residential_low	88.10	6.00	82.10	6.8	19.43	current_day	yes	1,355.94
H-016	North	residential_standard	88.90	11.80	77.10	13.3	26.37	current_day	no	2,033.10
H-017	South	residential_standard	96.08	4.29	91.79	4.5	26.20	current_day	no	2,404.79
H-018	East	residential_high	96.53	11.86	84.66	12.3	31.21	current_day	no	2,642.27
H-019	West	residential_standard	94.92	5.88	89.04	6.2	26.45	current_day	yes	2,001.82
H-020	North	residential_low	96.72	11.76	84.96	12.2	18.67	current_day	no	1,586.13
H-021	South	residential_standard	90.78	4.28	86.50	4.7	23.83	current_day	no	2,061.25
H-022	East	residential_low	92.53	11.43	81.10	12.3	18.72	current_day	no	1,518.21
H-023	West	residential_standard	87.90	5.89	82.02	6.7	25.27	current_day	no	2,072.57

Figure 7. Scheduled consolidated report file written by the Airflow publish_report task.

8.3 Lambda merge: provisional bills during the day

At capture time (sim day 5, 11:33) household H-001 had consumed 28.63 kWh with 1.58 kWh of solar according to the speed layer; applying its latest batch-validated tariff (17.81 LKR/kWh, from day 4) gives a provisional bill so far of LKR 481.81, next to its final bill of LKR 1,591.13 for day 4. The provisional total for all 24 households was LKR 15,491.04. Figure 8 shows the same endpoint for this household, queried a few seconds later (the figures grow as readings arrive).

```

GET http://localhost:8000/api/billing/projection?household_id=H-001

{
  "source": "lambda_merge(speed_layer consumption x batch_layer tariff)",
  "clock": {
    "sim_day": 5,
    "day_fraction": 0.546,
    "sim_time_of_day": "13:06",
    "daylight": true,
    "simulated_day_seconds": 300
  },
  "count": 1,
  "total_provisional": 560.68,
  "rows": [
    {
      "household_id": "H-001",
      "grid_zone": "South",
      "consumption_kwh": 34.403,
      "solar_kwh": 2.922,
      "reading_count": 78,
      "tariff_rate": 17.81,
      "billing_tier": "residential_low",
      "subsidy_flag": false,
      "tariff_sim_day": 4,
      "last_final_bill": 1591.13,
      "last_final_sim_day": 4,
      "net_consumption_kwh": 31.481,
      "solar_contribution_pct": 8.49,
      "provisional_bill_so_far": 560.68,
      "status": "provisional"
    }
  ]
}

```

Figure 8. GET /api/billing/projection — speed-layer consumption merged with the batch-layer tariff.

8.4 Speed vs batch reconciliation and data quality

Sim day	Raw events	Dup. removed	Speed kWh	Batch kWh	Drift %	Invalid tariff rows	Fallback tariffs	Billed (LKR)
0	3599	0	2236.16	2236.16	0.00	0	0	48,392.51
1	3003	0	1821.50	1821.50	0.00	0	0	38,471.54
2	1716	5	1409.65	1405.80	0.27	0	0	31,798.40
3	3669	6	2271.84	2268.20	0.16	0	0	49,030.64
4	3641	4	2244.16	2241.12	0.14	2	1	48,398.41

Over the run the batch layer read 15,628 events from the lake and removed 15 duplicates (days 2, 3, 4). These are redelivered readings that arrived in a later micro-batch than the original, so the speed layer's in-batch dedup could not see them and its running total counted them twice — a positive drift is exactly that over-count, and the batch layer's global dedup on event_id removes it from the final bill. Days with zero drift had no cross-batch duplicates. Tariff defects occurred on day(s) 3, 4: the invalid or duplicated rows were rejected and 1 household(s) were billed with their previous valid tariff instead. Days 1 and 2 also span the 3.5-minute producer outage of the failure drill (section 6.1), which lowers their consumption and bills but not their correctness: both layers saw the same, smaller set of events.

In the last 5 minutes before capture the speed layer processed 3679 events across 30 micro-batches, quarantined 45 (1.22%; bad_timestamp: 16, out_of_range: 12, missing_field: 10, negative_value: 7), dropped 3 in-batch duplicates, with an average event-to-sink lag of 5.3 s.

8.5 Alerts, orchestration and monitoring

Alerts raised during the run, by type: LOW_RENEWABLE $\times$ 8, NO_DATA $\times$ 2. The LOW_RENEWABLE alerts by zone were South $\times$ 5, West $\times$ 1, East $\times$ 1, North $\times$ 1: South, which has the least installed solar, drops below 10% at the start of simulated daylight every day, and the other zones triggered only while under that day's overcast spell. Every one auto-resolved within one or two 15 s checks once the next window recovered. The two

NO_DATA alerts are the failure drill of section 6.1. Because the rule has no minimum duration, a zone hovering around the threshold can raise short alerts; section 9 lists the fix.

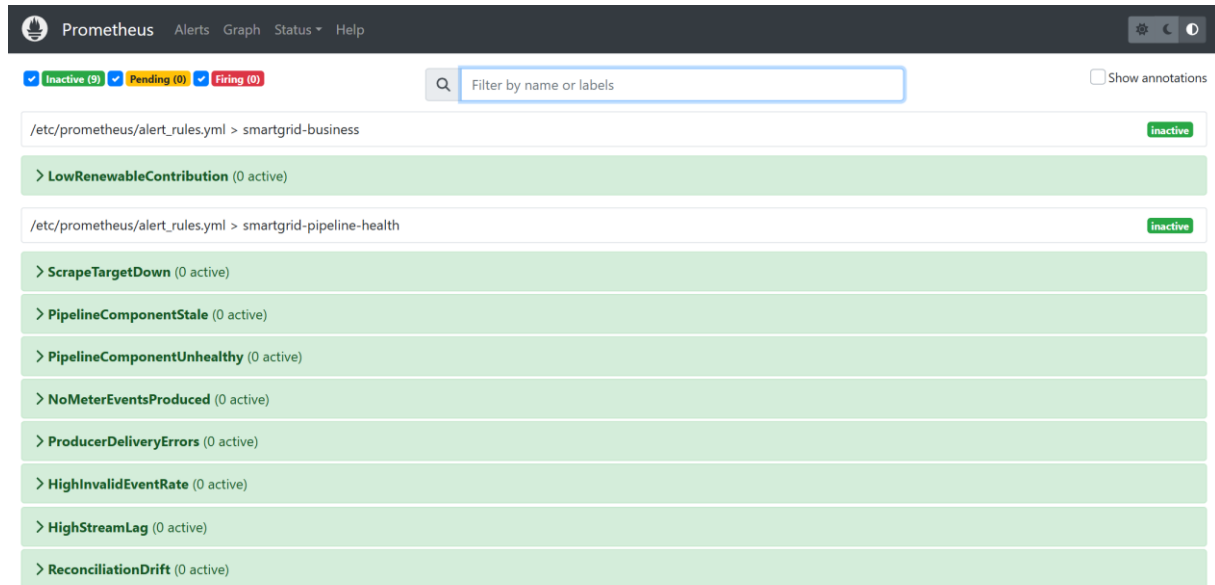


Figure 9. Prometheus: the pipeline's alert rules loaded and evaluated.

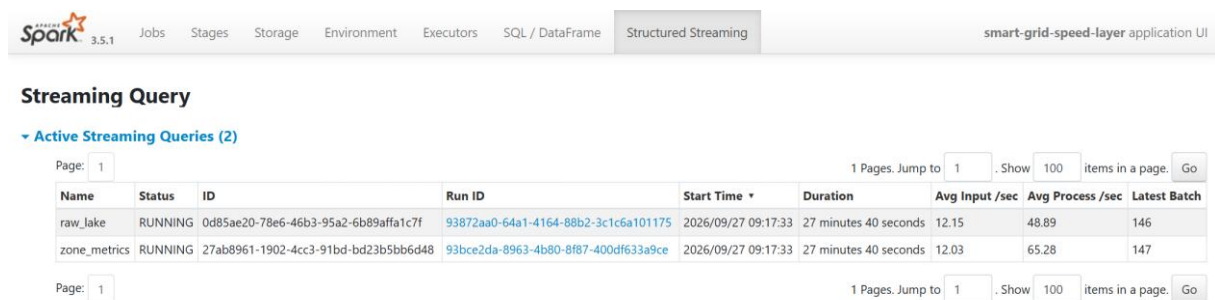


Figure 10. Spark UI: the two Structured Streaming queries (zone_metrics, raw_lake).

9. Limitations, Trade-offs and Production-Scale Changes

Area	Simplification in this project	What we would do at production scale
Kafka	1 broker, replication factor 1, no auth	≥ 3 brokers, RF=3, min.insync.replicas=2, idempotent producer, TLS/SASL, schema registry (Avro) for evolvable event schemas
Spark	local[2] in a container; batch Spark runs inside the Airflow container	Spark on Kubernetes/YARN; SparkSubmitOperator or KubernetesPodOperator; autoscaling
Speed-layer accuracy	At-least-once: in-batch dedup only; additive running totals can double count after a crash	Stateful dedup with watermark or transactional sinks for exactly-once; keep the batch layer as the source of truth either way
Late data	Events later than the 1-minute watermark are dropped from windows (still in the lake)	Tune watermark from observed lateness; re-run the batch day if late data arrives after billing
Airflow	Standalone, SequentialExecutor, SQLite metadata	Celery/Kubernetes executor with a Postgres metadata DB; data-aware scheduling (Datasets) triggered by the tariff file
Data lake	Local Docker volume, Parquet	S3/HDFS with a table format (Delta/Iceberg) for ACID appends, compaction and time travel
Serving	Single Postgres, no auth, dashboard polls every 5 s	Read replicas or a time-series store (TimescaleDB) for live metrics; API auth; push updates (WebSocket)
Observability	Home-grown rule engine + single Prometheus, no Grafana/Alertmanager;	Alert hysteresis (e.g. breach for 2 min, clear above 12%), Alertmanager routing/on-call, Grafana dashboards, OpenTelemetry tracing, central log store

Area	Simplification in this project	What we would do at production scale
	LOW_RENEWABLE has no minimum duration, so it can flap near the threshold	
Billing	Flat per-kWh tariff, one subsidy rule, no export credit	Tiered/time-of-use tariffs, net-metering credits, versioned tariffs and an audit trail
Simulation	24 households, 5-minute days	Real meter volumes need more partitions and a partitioning scheme by zone/date

What we would do differently. We would introduce a schema registry from day one (the ad-hoc JSON schema is the main source of dirty-data risk), use a table format such as Delta Lake so the batch layer could do atomic per-day overwrites directly in the lake, and move the renewable rule to a forecast-aware threshold (expected solar for the hour and weather) instead of a fixed 10%.

10. Conclusion

The platform ingests a continuous meter stream and a daily tariff extract, answers the business question in real time (grid load and renewable mix by zone) and per day (exact household bills and solar contribution), and shows both views plus their merge in one API and dashboard. Lambda was chosen because the use case combines a low-latency monitoring need with a correctness-critical, replayable daily computation over a naturally batch source, and the measured speed-versus-batch drift shows why the batch layer is needed. The pipeline is observable at every stage through structured logs, heartbeats, quality and lag metrics, seven alert rules and Prometheus.

Individual contributions

- **Sewvandi M.A.K. (EG/2021/4808):** Lambda vs Kappa architecture decision and overall system design; Spark Structured Streaming speed layer (cleaning, windowing, raw data lake); Spark batch layer (daily recompute, tariff join, billing, reconciliation); Airflow orchestration; PostgreSQL schema; report writing.
- **Peiris P.R.S. (EG/2021/4706):** smart-meter and tariff data simulators; Kafka ingestion setup; REST API and live dashboard; observability (structured logging, alert rules, Prometheus); Docker Compose setup; automated tests and results capture.